

# CS1033 Programming Fundamentals

# Laboratory Exercises

## Part A: Labs 1-5

BSc Engineering - Semester 1 (2023 Intake)  
June – October 2024

Department of Computer Science and Engineering  
Faculty of Engineering  
University of Moratuwa  
Sri Lanka

**Compiled by:**

Sanath Jayasena, Thanuja Ambegoda, Chandana Gamage, Gayashan Amarasinghe, Kalana Wijegunaratna, Piyumal Demotte, Haritha Jayasinghe, Ravin Gunawardena, Tarindu Jayatilaka, Thirasara Ariyaratna, Srikandabala Kogul, Kirishnni Prabagar, Budvin Edippuliarachchi, Himashi Rathnayake and Praditha Alwis.

© Dept of Computer Science & Engineering, University of Moratuwa

**Acknowledgements**

All help and support provided to make this document possible are acknowledged.

Sections of this document were based on:

- Python Tutorial by <https://www.tutorialspoint.com/>
- Introduction to Computing Using Python: An Application Development Focus by Ljubomir Perkovic.
- Python documentation at <https://docs.python.org/3/contents.html>
- C Programming Lab Exercises document for CS1033 used in previous years, contributed by several members of the Dept of Computer Science & Engineering. They had the following references:
  - The C Programming Language (ANSI C Edition) by Brian W Kernighan and Dennis M Ritchie, Prentice-Hall
  - 'Learn C in Three Days' by Sam A. Abolrous, BPB Publications & Programming in ANSI C, second edition by E. Balagurusamy, Tata McGraw-Hill.
- Programming problems from Instructors in the Dept of Computer Science & Engineering

**Date of this version: 24 June 2024**

# Table of Contents

## Contents

|                                                    |    |
|----------------------------------------------------|----|
| Pre-requisites .....                               | 4  |
| Pre-Lab Activity: Introduction to CS1033 Labs..... | 4  |
| Lab 1: Introduction to Python Programming.....     | 5  |
| L1.1 Using Python in Interactive Mode .....        | 5  |
| L1.2 Python Programs .....                         | 6  |
| L1.3 Developing Programs.....                      | 6  |
| L1.4 Syntax Errors and Bugs.....                   | 9  |
| L1.5 Comparing Python and C .....                  | 12 |
| L1.6 Practice Programs .....                       | 13 |
| Lab 2: Conditional Control Structures.....         | 14 |
| Lab 3: IDEs and Debuggers.....                     | 16 |
| Lab 4: Loop Control Structures .....               | 19 |
| Lab 5: Lists .....                                 | 20 |
| Lab 6: <to be added> .....                         | 22 |
| Lab 7: <to be added> .....                         | 23 |
| Lab 8: <to be added> .....                         | 24 |
| Lab 9: <to be added> .....                         | 25 |
| Lab 10: <to be added> .....                        | 26 |

## Pre-requisites

The following are assumed (these are prerequisites for CS1033 labs).

1. You have an account on the LearnOrg LMS (our Learning Management System at <https://lms.uom.lk/>) and you have logged on to it at least once.
2. You have registered for Semester 1 course modules, including CS1033, on the LMS.
3. You are familiar with the LMS.
4. You have logged on to our online course portal running on Moodle (at <https://online.uom.lk/>) at least once.
5. You have accessed the current offering of CS1033 on Moodle at least once.
6. You have basic Moodle skills as a student (can access resources; can complete activities like quizzes, forums, assignments; can manage your profile).

## Pre-Lab Activity: Introduction to CS1033 Labs

This pre-lab activity is to be completed during the first week of the semester. Note that it is very important for a student to complete this pre-lab activity in order to successfully proceed with the CS1033 during the semester and to achieve the intended learning outcomes. The CS1033 Labs (described starting on page 5 of this document) are to be completed by students one-by-one, generally on a weekly basis, starting from the second week of the semester.

---

### 1. Objectives

After completing this part successfully, you should be able to:

1. List / describe student responsibilities applicable to CS1033 lab sessions.
2. Identify the staff associated with CS1033 lectures and labs.
3. Install and use Python and Jupyter Notebook on your personal computer.

### 2. Procedure

**Part 1:** Attend the online Webinar on Zoom (details and schedule provided on Moodle) in which CS1033 staff members from the Department of Computer Science & Engineering (CSE) will help you to achieve Objectives (1) and (2) above, among others, as follows:

- Welcome you to CS1033.
- Introduce the staff associated with CS1033.
- Explain relevant rules, guidelines, and your responsibilities.

If you could not attend the webinar, you can later access the recordings of the session on Moodle.

**Part 2:** Follow the detailed online instructions on the CS1033 Moodle site to achieve Objective 3. If you have difficulties, do not worry; during the first week, we will conduct special online interactive sessions (physical sessions cannot be conducted due to the current situation), to help you to successfully complete Objective 3. The webinar session and the Moodle site will provide more details and schedules of the special interactive sessions. If you still have problems (i.e., you did not achieve the Objectives 1-3 above during the first week), then you should get in touch with the instructor-in-charge if your group and get their support.

# Lab 1: Introduction to Python Programming

## Objectives

- Edit a Python program.
- Execute Python programs on your preferred operating system using the Python Interpreter.
- Correct (fix) syntax errors and bugs, taking clues from error messages and warnings.
- Debug programs through program testing.
- Detect programming errors through program testing.
- Be familiar with a text editor on your preferred operating system.

## Requirements

- The Python Interpreter (Python version 3).

## Remarks

- This lab exercise will take more than one lab session (timetable slot).
- Lab 1 will not be evaluated.
- All the source codes and executables related to this lab session should be saved in a folder named **Lab1**.
- All the source codes related to this lab should be submitted to the respective lab activity on Moodle.
- All the source codes and diagrams related to the Exercises L1.3, L1.4 should be submitted to the respective lab activity on Moodle.

### L1.1 Using Python in Interactive Mode

To start the Python interpreter in *interactive mode*, type the command **python** at the command prompt or terminal.

When commands are read from the keyboard, the interpreter is said to be in interactive mode. In this mode it prompts for the next command with the primary prompt, usually three greater-than signs (>>>); for continuation lines it prompts with the secondary prompt, by default three dots (...). The interpreter prints a welcome message stating its version number and a copyright notice before printing the first prompt:

```
Python 3.11.3 (v3.11.3:f3909b8bc8, Apr 4 2023, 20:12:10) [Clang 13.0.0
(clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

After that, you can type commands and statements at the primary prompt. Some examples:

```
>>> 1 + 5
6
>>> 2 * 5
10
>>> len('aaa') + len('ttt')
6
>>> print("Hello World!\n")
Hello World!

>>> for i in range(3):
...     print(i, end=" ")
0 1 2
```

You can try many others. The interactive mode makes it easy to experiment with features of the language, or to test functions during program development. It is also a handy desk calculator.

But our objective here is to focus on the non-interactive use of Python, that is, to develop and run *Python programs* (also called *Python scripts*), which are sequences of Python commands.

## L1.2 Python Programs

Perform the following steps.

Step1: Type the Python program given below using a text editor such as **gedit** (or **Notepad** or **vi** or **emacs** or **Kwrite**) and then save it as **test1.py** in an appropriate directory.

```
# display Hello World

print("Hello world !\n")
```

Step 2: Open a new terminal or a command window. Now you can enter any command using the shell. Go to the directory where you saved the program source file **test1.py** using **cd** command.

Step 3: Next, to run the program saved in Step 1, enter the following command at the command/shell prompt.

```
>>> python test1.py
Hello world !
```

**Exercise:** Modify the above program to display the program text (source code).

*Hint:* Study the use of escape sequences (escape codes) `\n`, `\t` and `\"` with `print`.

## L1.3 Developing Programs

As described on the Section 1.1.1 of the Course Notes Part A, developing a program involves the following steps most of which are common to any problem-solving task.

1. Define the problem
2. Outline the solution
3. Develop an algorithm
4. Test the algorithm for correctness
5. Code the algorithm in a programming language to obtain a program
6. Ensure program has no syntax errors
7. Compile to generate translated code (optional)
8. Run the program
9. Test and debug the program
10. Document and maintain the program

The first four steps of identifying the problem and coming up with the algorithm is crucial for the success of the solution. Once the algorithm has been outlined and tested, any suitable programming language can be chosen to implement the algorithm. A successful programmer follows these steps to develop successful solutions using programming.

As an example, let's consider how solving a quadratic equation should be approached.

Step 1: Define the problem - Develop a program that can solve a quadratic equation  $ax^2 + bx + c = 0$  for  $x$  given the integers  $a$ ,  $b$  and  $c$ .

Step 2: Outline the solution – The solutions of a quadratic equation  $ax^2 + bx + c = 0$  is given by the quadratic formula,

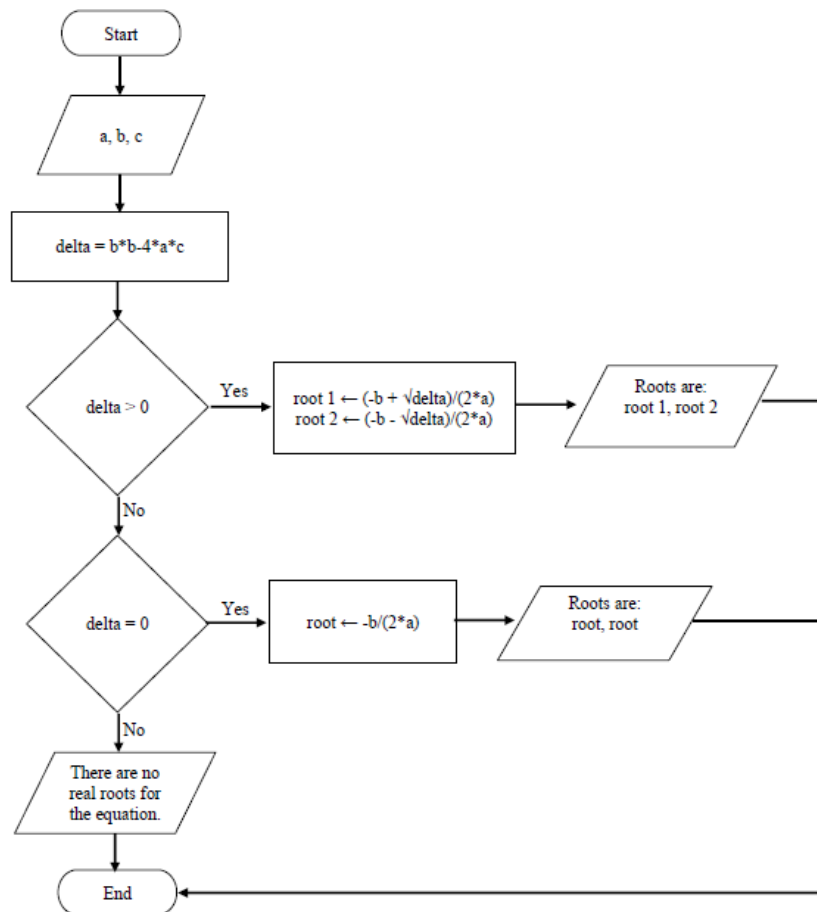
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

provided that  $b^2 - 4ac \geq 0$

Step 3: Develop an algorithm – Let's develop the algorithm using pseudocode.

```
BEGIN
INPUT a, b, c
delta ← b*b - 4*a*c
IF delta > 0 THEN
    root1 ← (-b+ √delta) / (2*a)
    root2 ← (-b- √delta) / (2*a)
    OUTPUT "Roots are:", root1, root2
ELSE IF delta = 0 THEN
    root ← -b / (2*a)
    OUTPUT "Roots are:", root, root
ELSE
    OUTPUT "There are no real roots for the equation."
ENDIF
END
```

Another important tool for developing algorithms is flow charts (introduced in the section 1.1.4 of Course Notes Part A).



Step 4: Test the algorithm for correctness – Consider a few quadratic equations that and try them with your algorithm designed in the Step 3. Check if it produces the expected answers for different scenarios.

*Hint: You can improve your algorithm during testing. See if you can improve the above algorithm.*

Step 5: Code the algorithm in a programming language to obtain a program – Now that you have a tested algorithm, you are free to choose a suitable programming language to implement your solution.

For example, If the Python programming language was chosen, below is how the above algorithm would be implemented:

```

import math

a = int(input("Enter a : "))
b = int(input("Enter b : "))
c = int(input("Enter c : "))
delta = b*b - 4*a*c
if delta > 0:
    root1= (-b - math.sqrt(delta)) / (2*a)
    root2= (-b + math.sqrt(delta)) / (2*a)
    print("Roots are: ", root1, root2)
elif delta == 0:
    root = -b/(2*a)
    print("Roots are: ", root, root)
else:
    print("There are no real roots for the equation.")
  
```

If the C programming language was chosen, below is how the above algorithm would be implemented:

```

#include <stdio.h>
#include <math.h>
int main( )
{
    int a, b, c, delta;
  
```

```

float root1, root2, root;
printf("Enter a: ");
scanf("%d", &a);
printf("Enter b: ");
scanf("%d", &b);
printf("Enter c: ");
scanf("%d", &c);
delta = b*b - 4*a*c;
if (delta > 0)
{
    root1 = (-b - sqrt(delta)) / (2*a);
    root2 = (-b + sqrt(delta)) / (2*a);
    printf("Roots are: %.1f %.1f\n", root1, root2);
}
else if (delta == 0)
{
    root = -b/(2*a);
    printf("Roots are: %.1f %.1f\n", root, root);
}
else
{
    printf("There are no real roots for the equation.");
}
return 0;
}

```

Steps 6,7,8,9 and 10 – This lab note, and the Course Notes Part A and B will guide you through with these steps.

#### L1.4 Syntax Errors and Bugs

Programmers, being humans, often end up writing source code with errors.

Some of these errors result in source code that does not conform to the *syntax rules (grammar rules)* of the Python language and are caught by the interpreter. Some others however, surface only later and cause programs to fail at performing their intended tasks. Such defects in programs are called *bugs* and the process of eliminating them is called *debugging*.

Perform the following steps.

Step 1: Shown below is a Python program intended to compute the roots of the quadratic equation  $ax^2 + bx + c = 0$  provided that  $b^2 - 4ac \geq 0$ , given integers  $a$ ,  $b$  and  $c$ .

Type the following program exactly as given using a text editor and then save it as **test2.py**.

```

a = int(input("Enter a :"))
b = int(input("Enter b : "))
c = int(input("Enter c : "))
delta = b*b - 4*c
if delta < 0:
    print("complex roots !")
else:
    t= math.sqrt(delta)
    r1= (-b -t) / (2*a)
    r2= (-b +t) / (2*a)
    print("Roots are: ", r1, r2)      # display the roots

```

Step 2: Now, to run the program, enter the command **python test2.py** at the command/shell prompt. The text below shows something like what you will get next!

```
$ python test2.py

File "test2.py", line 1
    a = int(input("Enter a :))
           ^
SyntaxError: unterminated string literal (detected at line 1)
```

The above message is reported by the Python interpreter when it processes the source file, **test2.py**. This is because of the identified syntax error in the source code.

The way errors are reported depends on the interpreter and the environment. Notice that the interpreter reports the file name and the line number where it found the syntax error.

Step 3: Let's correct the error reported above. Notice that the error message is concerned with line 1.

Open the file **test2.py** in an editor and go to line 1. Try to figure out what is wrong and correct it. You may also want to take into consideration the following part of the error message:

```
unterminated string literal (detected at line 1)
```

After fixing line 1, run **test2.py** again.

If you corrected the error on line 1 properly, you should now get a message similar to the following:

```
File "test2.py", line 6
    print("complex roots !")
    ^
IndentationError: expected an indented block after 'if' statement on line 5
```

This is due an indentation error on line 6. In Python, a block of code belonging to an **if** clause should be indented to the right (e.g., with a tab). Correct this error.

After correcting line 6, go back and try to run **test2.py** again. If you correctly followed all the above instructions, you should not get any more error messages.

Step 4: You may run the program by entering the following command at the prompt. Press **Enter** key after typing the values for the coefficients **a**, **b**, and **c**.

```
$ python test2.py
Enter a : 1
Enter b : -7
Enter c : 12
```

Now you will see the following error message.

```
Traceback (most recent call last):
  File "test2.py", line 8, in <module>
    t= math.sqrt(delta)
NameError: name 'math' is not defined
```

Here the interpreter has reported an error. The cause is the **math.sqrt** function. Whenever any mathematical function (e.g., sqrt, sin, cos) are used, the **math** module needs to be imported. So, the above error can be corrected by inserting an "import math" statement at the beginning of your program as follows.

```
import math
a = int(input("Enter a :"))
b = int(input("Enter b :"))
c = int(input("Enter c :"))
....
```

Now run the program again. Press **Enter** key after typing the values for **a**, **b**, and **c**.

```
$ python test2.py
Enter a : 1
Enter b : -7
Enter c : 12
Roots are: 3.0 4.0
```

Here 3.0 and 4.0 are the correct roots for the given values for a, b and c.

Try the program with several more sets of values for **a**, **b**, and **c**. Such sets of values used for testing a program are known as *test cases*.

Step5: When developing a program, syntax errors are probably not that difficult to correct. Referring to the previous step, did the program work correctly? If you tried a test case where **a** is not equal to 1, you would notice that it gives wrong answers. Here is an example:

```
$ python test2.py
Enter a : 2
Enter b : -14
Enter c : 24
Roots are: 1.0 6.0
```

Step 6: Had you paid attention to the expression that computes the value of `delta`, you would have noticed a fault. The expression calculates the discriminator  $\Delta = b^2 - 4ac$  of the quadratic equation. However, it fails to use the proper formula in calculation. Here is the expression:

```
delta = b * b - 4 * c;
```

This is an example of a programming error or *bug*. Detecting bugs in programs can be hard, though it is easy in this case. *Code inspection* (where one or more people read someone else's program line by line) and *program testing* (running the program on sample input data - like we did above) are two common techniques in software engineering.

Now, go ahead and correct the expression. Then run the program again with the test case for which the program failed earlier and see whether it works correctly now.

Step 7: Next, try the following test case against your program and see how it behaves when **a** is equal to zero.

```
$ python test2.py
Enter a : 0
Enter b : -7
Enter c : 12
Traceback (most recent call last):
  File "test2.py", line 10, in <module>
    r1= (-b -t) / (2*a)
ZeroDivisionError: float division by zero
```

When a program performs an illegal operation at *run-time* (while it is executing), it is called a *run-time error*. Here, the program performed a division by zero for the input data set **{0, -7, 12}**. Looking at the source code, if we enter a value of 0 for **a**, this leads to a division by zero later in the program. One possible solution is to check the value we input for coefficient **a** using an **if** clause. Then, since we check before we divide, even if we enter a value of zero, it does not lead to a division by zero.

Step 8: See below for the program which addresses all the above issues. Test the program with all the input combinations that we used before.

```
import math
a = int(input("Enter a : "))
b = int(input("Enter b : "))
c = int(input("Enter c : "))

if a==0:
    print("Coefficient a should be non-zero.\n" )
else:
```

```

delta = b*b - 4*a*c
if delta < 0:
    print("complex roots !")
else:
    t= math.sqrt(delta)
    r1= (-b -t) / (2*a)
    r2= (-b +t) / (2*a)
    print("Roots are: ", r1, r2)    # display the roots

```

Step 9: Open Lab 1 quiz on the Moodle and copy the Python program to the text editor and check your answer.

### L1.5 Comparing Python and C

To get a basic understanding on the differences between the Python and C languages (as well as their language implementations), we will do the following. You should first read Section 1.4 in the Course Notes. Next perform the following steps.

As you are attempting this lab on your personal computer, we do not expect you to install a C compiler. Therefore, we will use the Lab 1 quiz activity on Moodle to compare the two languages.

Step 1: Open the Lab 1 quiz on the Moodle and type in the "Hello, World!" C program in Section 1.4 in Course Notes. Check the program and see if it passes the test case.

Step 2: In the next question, type in the following C program that computes the roots of the quadratic equation  $ax^2 + bx + c = 0$  provided that  $b^2 - 4ac \geq 0$ , given integers  $a$ ,  $b$  and  $c$ . This C program was developed by referring to the Python program **test2.py** above of this lab. Check the program and see if it passes the test case.

```

#include <stdio.h>
#include <math.h>
int main( )
{
    int    a, b, c, delta;
    float  t, r1, r2;
    printf("Enter a: ");
    scanf("%d", &a);
    printf("Enter b: ");
    scanf("%d", &b);
    printf("Enter c: ");
    scanf("%d", &c);
    if (a==0)
        printf ("Coefficient a should be non-zero.\n");
    else {
        delta = b*b - 4*a*c;
        if (delta < 0)
            printf("complex roots !\n");
        else {
            t = sqrt(delta);
            r1 = (-b - t) / (2*a);
            r2 = (-b + t) / (2*a);
            printf("Roots are: %.1f %.1f\n", r1, r2);
        }
    }
    return 0;
}

```

Step 3: What are the main differences between the two languages and how they are implemented? Based on the Section 1.4 of the lecture notes, the C program is compiled into an executable code and stored on disk. What do you say about the lengths of the C program and the Python program, intended for the same task?

If you are interested in compiling a C program, you can try the following steps to compile and run a C program on your computer. You will have to have installed a compatible C compiler on your operating system.

Optional Step 1: Save the above C program using a text editor and save it as **test2.c**. Compile the C program in **test2.c** file as follows. This will produce the executable file **a.out**, if there are no compilation or linking errors. (Try compiling without "-lm" and see).

```
$ gcc test2.c -lm
```

Optional Step 2: Run the executable as follows:

```
$ ./a.out
```

## **L1.6 Practice Programs**

1. To get more understanding on the basics of Python, practice commands and programs in Chapter 2 of the CS1033 Course Notes.
2. To get more understanding on Operators and Expressions in Python, practice commands and programs in Chapter 3 of the CS1033 Course Notes.

## Lab 2: Conditional Control Structures

### Remarks

- This lab will not be evaluated.
- For drawing flowcharts, you can use an online tool such as lucidchart or you can draw on a paper and upload the photograph of the diagram.
- You can either use a simple text editor or an IDE such as IDLE.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L2.E1, L2.E2 and L2.E3 should be submitted to the respective lab activity on Moodle.

### L2.1 Practice Programs

1. To get a better understanding on Conditional Control Structures, practice programs in Chapter 4 of the given CS1033 Course Notes.

### L2.2 Exercises

**Exercise L2.E1** - Develop a program to input three angles of a triangle and output the type of the triangle. First, you must check whether the given angles form a triangle or not. If they do form a triangle then output the type of the triangle as “Right angled”, “Obtuse angled” or “Acute angled” triangle. If the angles do not form a triangle you can output “Angles do not form a triangle”. You must draw a flowchart to express the algorithm of your program.

Input: Enter angle 1: 45  
Enter angle 2: 90  
Enter angle 3: 45  
Output: Right angled

**Exercise L2.E2** – Given a date as a triplet of numbers (**y**, **m**, **d**), with **y** indicating the year, **m** the month ( $m = 1$  for January,  $m = 2$  for February, etc.), and **d** the day of the month, the corresponding day of the week **f** ( $f = 0$  for Sunday,  $f = 1$  for Monday, etc), can be found as follows:

- a) if  $m < 3$  let  $m = m + 12$  and let  $y = y - 1$
- b) let  $a = 2m + 6(m + 1) / 10$
- c) let  $b = y + y/4 - y/100 + y/400$
- d) let  $f_1 = d + a + b + 1$
- e) let  $f = f_1 \bmod 7$
- f) stop

Develop a program that reads a date as three numbers separated by white spaces and outputs the corresponding day of the week. All divisions indicated above are integer divisions. You must draw a flowchart to express the algorithm of your program.

**Exercise L2.E3** – Develop a program to input the units of electricity (number of kWh) consumed monthly and output the calculated electricity bill for that month. The charges differ according to the following domestic purpose plan from Ceylon Electricity Board. You must draw a flowchart to express the algorithm of your program.

- If the consumption is between 0-60 kWh per month the following tariffs will be applicable.

| Monthly Consumption kWh | Unit Charge (Rs/kWh) | Fixed Charge (Rs/month) |
|-------------------------|----------------------|-------------------------|
| 0-30                    | 30                   | 400.00                  |
| 31-60                   | 37                   | 550.00                  |

- If the consumption is above 60 kWh per month the following tariffs will be applicable.

| Monthly Consumption kWh | Unit charge (Rs/kWh) | Fixed charge (Rs/month) |
|-------------------------|----------------------|-------------------------|
| 0-60                    | 42                   | -                       |
| 61-90                   | 42                   | 650                     |
| 91-120                  | 50                   | 1500                    |
| 121-180                 | 50                   | 1500                    |
| >180                    | 75                   | 2000                    |

**e.g** - If the consumption is 45kWh, the calculation would be as follows:

$$\text{Monthly Bill} = 30 \text{ kWh} * 30 \text{ Rs/kWh} + (45-30) \text{ kWh} * 37 \text{ Rs/kWh} + \text{Rs } 550 = \text{Rs } 2005$$

Note that the fixed cost in the calculation (Rs 550 in the above example) is the fixed cost pertaining to the total consumption. The fixed cost is **not cumulative**.

## Lab 3: IDEs and Debuggers

### Objectives

- This lab will not be evaluated.
- Develop Python programs using an *Integrated Development Environment* (IDE).
- Correct syntax errors and bugs by taking clues from error messages.
- Use a debugger to debug a program.

### Prerequisites

- Students are expected to be familiar in developing Python programs on their preferred platform.

### Requirements

- The *Python IDLE* IDE.

### Remarks

- This lab will not be evaluated.
- All the source codes and executables related to this lab session should be saved in a folder named **Lab3**.
- All the source codes related to the lab should be submitted to the respective lab activity on Moodle.

*Integrated Development Environment* (IDE) is a Graphical User Interface (GUI) based application or a set of tools that allows a programmer to write, compile/interpret, run, edit and in some cases test and debug within an integrated, interactive environment. These tools provide a lot of features that make programming a fun activity. It may take some time for you to understand each and every feature provided by the IDE but it will certainly enhance your productivity. Some of these tools go beyond conventional program development and testing and even provide facilities to manage various documents related to the project or source code to keep track of different versions of the source code. *Python IDLE* is a popular IDE for developing Python programs. From this lab session onward, you can use Python IDLE for program development.

### L3.1 Using Python IDLE

To launch Python IDLE, type the command `idle` at the command/shell prompt on a terminal on Linux or by accessing Python IDLE through the windows start menu.

Then the Python IDLE window appears.

### L3.2 Your First Program with IDLE

You can work interactively with IDLE, the same way you would interact with the Python interpreter running on a terminal. You can also run Python programs (scripts) using this IDE. To create a new file you should follow the following sequence.

#### File → New File

Then a new editor window will appear, and you can write your Python program in that editor.

Now enter the following line in the editor:

```
print("Hello world !\n")
```

Next it is time to run your Python program using IDLE. First you need to save your program. Use **File → Save** and select a folder on your computer.

Next, to run your program, either select **Run → Run Module** or press the function key **F5**.

If your program gets successfully executed, you should see the following message appearing in the main IDLE window (Python Shell).

```
Hello, World !
```

### L3.3 Locating Errors in a Program

Let us write a program to compute the factorial of a given number.

Step 1: Create a new Python file named **FactIDE.py**. Then type the following program exactly as shown.

```
import math
num = int(input("Enter number to find factorial :"))
fact=0
for i in range (1, num+1):
    fact = fact*i
print("Factorial of ", num, " is ", fact)
```

Step 2: Run the program by pressing **F5** or using **Run → Run Module**.

Executing the program with 5 as the input will display:

```
Enter number to find factorial: 5
Factorial of 5 is: 0
```

You will get something like above. But what is the factorial of 5? Is it 0? Although your program runs, you get the wrong output which means you have a logical error (bug). These types of errors can be tracked by debugging a program.

### L3.4 Debugging a Program

Now let us try to find a bug in the above program using a Debugger. You can refer to the video available on the Moodle course page to learn how to use the Python debugger.

Step 1: Go back to the source code and place a **Breakpoint** on the line with the expression.

`fact = fact*i`. To place a Breakpoint, move your mouse pointer to the relevant line and right click on the line and select **Set Breakpoint** from the sub menu. Then you should see the line highlighted in yellow colour. To clear a Breakpoint right click on the line again and select **Clear Breakpoint** from the sub menu.

Step 2: To debug your program select **Debug → Debugger** from the IDLE main window. Then you will see the Debug control window on the screen.

Step 3: Go back to the source code and run the program by pressing **F5** or selecting **Run → Run Module**. Then you will see the Debug control window appearing again on the screen.

Click on the **Go** button in the Debug control window to start the execution.

When the program requests you to enter a number to find the factorial, type 5 and press the Enter key. Then the program *will execute up to the breakpoint and halt*.

Step 4: In the Debug Control window you will see four checkboxes showing the Stack, Locals, Globals and the Source.

Locals shows the current values of the local variables. Globals shows the current values of the global variables.

Step 5: Click on the **Go** button.

Notice that the value of variable **i** gets changed but not the values of the other variables.

Step 6: Again, select **Run → Run Module**. You will still see only variable **i** is getting modified in the iterations of the for loop whereas variable **fact** should also get modified. Click on the Go button until you complete all the 5 iterations of the for loop.

Now you should be able to understand that there is some error in the statement **fact=fact\*i** which is supposed to update the value of variable **fact** inside the for loop. However you did observe that only variable **i** is getting updated. Then the error should be with the value of **fact**. Do you remember that the starting value of **fact** is **0** ? If not, use the Debugger again and verify that **fact** is always **0**. So regardless of the value of **i**, the multiplication always results in **0**. This is the bug in the program.

To debug (to fix the error), change the initial value of variable **fact** to **1** and **Run** your program again. Now enter 5 and verify that you get the correct answer. Test your program with several other inputs.

## Lab 4: Loop Control Structures

### Remarks

- This lab session will **not be** evaluated.
- First practice the programs mentioned in L4.1. You do not have to submit them.
- You can either use a simple text editor or an IDE such as IDLE.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L4.E1, L4.E2 and L4.E3 should be submitted to the respective lab activity on Moodle.

### L4.1 Practice Programs

1. To get a better understanding on Loop Control Structures, practice programs in Chapter 5 of the given CS1033 Course Notes.

### L4.2 Exercises

**Exercise L4.E1** - Develop a program that takes as input a series of positive integers and outputs whether each is a prime. You must draw a flowchart to express the algorithm of your program.

The program should terminate if a negative integer is given as the input. A prime number is a number that is divisible by only 1 and itself. However, 1 is not considered a prime number. Execution of your program should produce something like the following:

```
$ python prime.py
1
non-prime
2
prime
3
prime
4
non-prime
5
prime
-1
```

**Exercise L4.E2** - In number theory, an *abundant number* is a number for which the sum of its proper divisors is greater than the number itself. The integer 12 is the first abundant number. Its proper divisors are 1, 2, 3, 4 and 6 which sums up to 16. So, the total is greater than the number 12. The integer 14 is not an abundant number. Its proper divisors are 1, 2 and 7 which sums up to 10. So, the total is not greater than the number 14.

Develop a Python program to take as input a positive integer  $n$  greater than 1 and output the number of abundant numbers from 2 to  $n$  inclusive. Output 'Invalid Input' for inputs less than 2. You must draw a flowchart to express the algorithm of your program.

Input:

```
Input number: 15
```

Output:

```
Number of abundant numbers from 1 to 15 is 1
```

**Exercise L4.E3** - Encryption is the process of encoding information to prevent anyone other than its intended recipient from viewing it. Develop a Python program to encrypt a message. The message will be encrypted using a given *base*. You must obtain the ASCII value of each character of the message and convert that number into the given base. After converting values of all the characters, aggregate (concatenate) those values in the character order and show it as the encrypted message. You must draw a flowchart to express the algorithm of your program. An example is given below.

Note: ( $1 < \text{base} \leq 10$ )

Input: Enter message: Welcome to CSE

```
Enter base: 4
```

Output: 111312111230120312331231121120013101233200100311031011

## Lab 5: Lists

### Remarks

- **Exercises L5.E1, L5.E2, L5.E3 and L5.E4** in this lab session will be **evaluated**.
- First practice the programs mentioned in L5.1. These will not be evaluated, and you do not have to submit them.
- You **must** draw and submit **flowcharts** to express the algorithms of each of your programs. If you **do not submit**, then the submission will be incomplete and **will not be accepted**.
- All the source codes and diagrams related to the Exercises L5.E1, L5.E2, L5.E3 and L5.E4 should be submitted to the respective lab activity on Moodle.

### L5.1 Practice Programs

1. For basic understanding on Lists, practice programs in Chapter 6 of the CS1033 Course Notes.

### L5.2 Exercises

**Exercise L5.E1** - Develop a program to find and display the minimum and the maximum among 10 numbers separated by white spaces (see the example) entered from the keyboard. You must use a list to store the numbers entered. The numbers can be negative or non-integers. You must draw a flowchart to express the algorithm of your program. An example would be as follows:

```
Input:  5    7.8    9.6    54    3.4    1.2    3    7    8.8    5
Output: Minimum = 1.2
        Maximum = 54
```

```
Input:  4    -6    5.2    -3    4    2.4    35    734    9    -35.3
Output: Minimum = -35.3
        Maximum = 734
```

**Exercise L5.E2** - Develop a program to read the names of two sports that you and your friends love to play and watch. Then generate all sentences where the subject is in ["I", "We"], the verb is in ["play", "watch"] and the object is in the two sports. Use lists to store the words and generate the sentences by iterating through the lists using deeply nested loops. You must draw a flowchart to express the algorithm of your program.

An example:

```
Input:  cricket football

Output: I play cricket.
        I watch cricket.
        I play football.
        I watch football.
        We play cricket.
        We watch cricket.
        We play football.
        We watch football.
```

**Exercise L5.E3** Suppose there are 4 students each having marks of 3 subjects. Develop a program to read the marks from the keyboard and calculate and display the total marks and average mark (rounded off to one decimal point) of each student. Use a 2D (two-dimensional) list to store the marks. You must draw a flowchart to express the algorithm of your program. An example would be as follows:

```
Input (Enter the marks of four students, on four rows):
50    60    80
60    75    90
30    49    99
66    58    67
```

Output (Total marks and average marks of four students):

Total: 190 Average: 63.3  
Total: 225 Average: 75  
Total: 178 Average: 59.3  
Total: 191 Average: 63.7

**Exercise L5.E4** - Develop a program to input a matrix with any dimension and output the transpose of that matrix. You must draw a flowchart to express the algorithm of your program. You should stop accepting the rows when -1 is entered as the input. Use a 2D (two-dimensional) list to store the matrix. You should handle the exceptions such as checking the invalid rows with an inconsistent number of elements. An example:

Input:

1 2 3 4 5 6  
2 3 4 5 6 7  
3 4 5 6 7 8  
4 5 6 7 8 9  
-1

Output:

1 2 3 4  
2 3 4 5  
3 4 5 6  
4 5 6 7  
5 6 7 8  
6 7 8 9

Note: Print “**Invalid Matrix**” as the error message for invalid rows with an inconsistent number of elements. Print “**Error**” for any other exceptions you are handling.

## **Lab 6: <to be added>**

## **Lab 7: <to be added>**

## **Lab 8: <to be added>**

## **Lab 9: <to be added>**

## **Lab 10: <to be added>**